SECURITY

SECURITY REVIEW · PREPARED FOR

E280 ExternalLp

DATE

18.08.2026

CONDUCTED BY

Radev, Lead Security Researcher

Blckhv, Lead Security Researcher

COMMIT

740ec13f

REMEDIATION

74598792

01 ABOUT / DISCLAIMER / RISK CLASSIFICATION

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram @blckhv.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

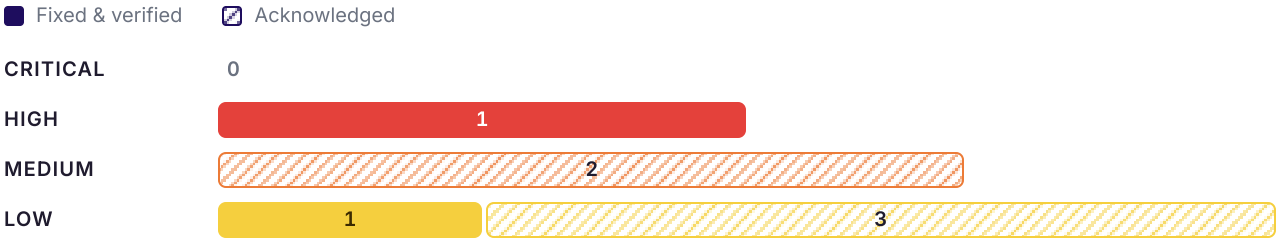
- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS	RESOLVED	HIGH SEVERITY RESOLVED
7	2 of 7 · 29%	1 of 1 · 100%

SEVERITY DISTRIBUTION



CLIENT	REPOSITORY	METHODS
E280 ExternalLp	Private	Manual review, AI Audit

CONTRACTS IN SCOPE

E280ExternalLpVault.sol	interfaces/IE280ExternalLpVault.sol
interfaces/INonfungiblePositionManager.sol	interfaces/IV3SwapRouter.sol
interfaces/IWETH.sol	interfaces/IWhitelistRegistry.sol

TIMELINE · AUGUST 2026

KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
11.08.2026	13.08.2026	18.08.2026	18.08.2026

STARTING COMMIT HASH	740ec13f33217b5b6c196df6aeb0954cd3c1e908
FINAL COMMIT HASH	74598792ec24fc9cef9589207285345cf9a76a43

03 PROJECT SUMMARY

What E280 ExternalLp is

The E280 External LP Vault is an ERC20 vault that manages a single Uniswap V3 liquidity position on behalf of both its share holders and the protocol. Users deposit token pairs (or E280, which is split and swapped into the pool tokens) and receive shares that are proportional claims on the user-owned portion of the position; a separately tracked protocol-owned portion accrues from deposit, withdraw, and compounding fees and never dilutes share holders. Whitelisted keepers compound collected trading fees and top up protocol liquidity, while the owner initializes, rebalances, and ultimately sunsets the position.

SECURITY POSTURE

BEFORE REMEDIATION 740EC13F

Fee accounting and keeper incentives

H-01 · L-04

Fee-batch construction and keeper-facing fee logic could tax idle principal rather than only realised fees, letting a whitelisted keeper convert protocol liquidity into its own profit, and the tiered deposit fee was bypassable through a lower-fee path.

Protocol exit and sunset liveness

M-01 · L-03

The sunset path could be permanently bricked by a fee recipient that cannot receive a transfer, locking all protocol-owned liquidity, and E280 sent to the vault after sunset - against a hard-coded, multi-chain-shared address - would be stranded.

Share pricing and deposit accounting

M-02 · L-02

Share price excluded value the vault already controlled, so a deposit placed one transaction before a realisation event could capture a share of it, and depositE280 credited the amount requested rather than the amount actually received while the tax-exemption prerequisite went undocumented.

Rebalance mechanics

L-01

Rebalance removed 100% of the vault's own liquidity before swapping through the same pool, worsening execution against the position it had just vacated.

AFTER REMEDIATION 74598792

Following the review, the E280 team addressed the highest-impact issue and one accounting/documentation finding, and formally acknowledged the remainder as accepted risks or intended design. The High-severity compound fee-batch issue (H-01) was fixed so that only realised fees - not idle principal - are taxed, removing the keeper's ability to divert protocol liquidity. The deposit-crediting and undocumented tax-exemption finding (L-02) was also resolved. The two Medium findings (M-01, M-02) and the remaining Low findings (L-01, L-03, L-04) were acknowledged: the team accepted the trusted-owner and configuration assumptions behind them and documented the operational requirements (fee-recipient safety, the E280 tax-exemption prerequisite, and keeper conduct) as deployment and monitoring obligations.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01	**Constrain and monitor whitelisted keeper actions**	compound and protocolDepositE280 are keeper-gated and move protocol liquidity. Keep the keeper whitelist minimal, monitor these calls on-chain, and alert on unexpected fee batches, swap sizes, or incentive claims so keeper misbehaviour is detected promptly.
R-02	**Harden fee-recipient and address configuration for exit paths**	sunset and the fee-distribution paths transfer to the genesis and buy-burn addresses and to a hard-coded E280 address. Use recipients that can always receive transfers (avoid contracts that may revert), verify the E280 address per target chain, and rehearse the sunset flow on a fork before mainnet use so liquidity can always be recovered.
R-03	**Add price-manipulation protections around swaps**	All swaps rely on caller-supplied minimum-output values for slippage protection with no on-chain oracle or TWAP reference. Consider bounding swap execution against a TWAP and continuing to enforce strict minimums on every keeper- and owner-initiated swap to reduce exposure to pool manipulation.

06 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
contracts/E280ExternalLpVault.sol	The core vault: an ERC20 share token that owns and manages a single Uniswap V3 position. Handles user deposits (pair, single-sided, and E280), withdrawals, keeper-driven fee compounding and protocol liquidity top-ups, and the owner-driven initialize / rebalance / sunset lifecycle, tracking the protocol-owned share of liquidity separately from user shares.

06 CONTRACTS & ROLES

Privileged roles

Owner.

Ownable2Step admin. Controls the vault lifecycle (initialize, rebalance, sunset), fee and allocation parameters (setProtocolFeeBps, setAllocationBps, setDepositSettings), the pause switch (setPaused), and non-core token rescue (rescueERC20). A fully trusted role.

Whitelisted keepers.

Addresses approved in the whitelist registry. May call compound (realise and reinvest trading fees) and protocolDepositE280 (deploy accumulated E280 into protocol-owned liquidity, earning an incentive). A semi-trusted operational role.

Genesis address.

Receives the fixed 8% genesis cut of every fee batch and protocol exit, and can reassign itself via setGenesisAddress. If lost, its allocations become unrecoverable.

Buy-burn address.

Immutable recipient of the buy-burn cut of fee batches and protocol exits, set at deployment and unchangeable thereafter.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	`compound` allocates the vault entire token balance as a fee batch, so idle principal is taxed and a whitelisted keeper can convert protocol liquidity into its own profit in one transaction	HIGH	FIXED
M-01	A fee recipient that cannot receive a transfer permanently bricks `sunset`, locking all protocol-owned liquidity	MEDIUM	ACKNOWLEDGED
M-02	Share price excludes value the vault already controls, so a deposit placed one transaction before any realisation event captures a share of it	MEDIUM	ACKNOWLEDGED
L-01	`rebalance` removes 100% of its own liquidity before swapping through the same pool	LOW	ACKNOWLEDGED
L-02	`depositE280` credits the amount requested rather than the amount received and the tax-exemption requirement is undocumented	LOW	FIXED
L-03	E280 received after `sunset` is permanently locked and the address is a constant across a multi-chain deploy config	LOW	ACKNOWLEDGED
L-04	The 300 bps `depositE280` fee is bypassable at 50 bps	LOW	ACKNOWLEDGED

HIGH SEVERITY · 1 FINDING

H-01 HIGH FIXED

`compound` allocates the vault entire token balance as a fee batch, so idle principal is taxed and a whitelisted keeper can convert protocol liquidity into its own profit in one transaction

Two defects. Neither does much alone.

`E280ExternalLpVault.sol#_deposit()` is the only path that gives back what Uniswap didn't consume:

```
 if (used0 < amount0) TOKEN_0.safeTransfer(msg.sender, amount0 - used0);
if (used1 < amount1) TOKEN_1.safeTransfer(msg.sender, amount1 - used1);
```

`initialize()`, `protocolDepositE280()`, `compound()` and `rebalance()` all throw away `used0` / `used1`. Whatever the position didn't take stays in the vault belonging to nobody and `rescueERC20()` blocks the pool tokens so it can't be swept.

This isn't a corner case. Uniswap consumes at exactly the position current ratio, while `protocolDepositE280` buys at a ratio the caller picks off-chain. When the position is out of range, which is ordinary V3 behaviour, it can only absorb one token and everything bought of the other side is stranded.

The second defect is that `E280ExternalLpVault.sol#compound()` runs the fee allocation over balances instead of over the fees it just collected:

```
 (uint256 fees0, uint256 fees1) = _collectFees();
uint256 balance0 = TOKEN_0.balanceOf(address(this));
uint256 balance1 = TOKEN_1.balanceOf(address(this));
...
(uint256 amount0, uint256 amount1) = _distributeTokenFees(postSwap0, postSwap1);
```

`fees0` / `fees1` only make it into the event. `rebalance()` and `sunset()` both pass the collected fees into that same helper, so `compound()` is the only caller that passes balances. Idle principal gets taxed as if it were fee income: 8% to genesis, `buyBurnBps` to buy-burn, then the protocol/user split on the rest.

Put together they hand a whitelisted keeper a lever. `ReentrancyGuard` is per-call rather than global, so a keeper contract can chain four vault calls in one transaction. `protocolDepositE280` takes `token0RatioBps` with no on-chain relation to the position ratio, so the keeper manufactures residue on demand out of the protocol E280 and recycles it through `compound()` while holding most of the shares.

IMPACT

The passive leak needs no malice and fires on every compound. Park 1,000 TOKEN_0 and 100 TOKEN_1 of pure principal with zero fees owed on the NFT, call `compound()` once and exactly 160 and 16 leave for genesis and buy-burn. That is 16% of principal.

The residue being taxed isn't dust. With the position out of range and the keeper picking a sane 50/50 split, one `protocolDepositE280` strands **18,115 tokens, 15.7% of user TVL**.

The skim also repeats. As long as fee income keeps supplying the missing token, each cycle re-taxes whatever survived the last: 10,000 → 8,402 → 5,936 → 3,531, so 65.3% reaches genesis and buy-burn after six cycles. Two limits on that: the re-tax only fires when enough of the other token is present and once only the unusable side is left `compound()` reverts instead of skimming again.

H-01 · CONTINUED

The keeper path is straight theft. Against an honest baseline, protocol-owned liquidity ends at 70,477 versus 27,815 attacked, so 42,662 units are gone and the keeper takes +9,472 tokens*. One transaction, capital returned inside it, repeatable every `interval`.

Nothing in the vault can stop a keeper that turns hostile. `WL_REGISTRY` is `immutable` with no setter, `WhitelistRegistry` carries its own owner and `compound()` isn't covered by `setPaused`. The only remedy is `sunset()`.

RECOMMENDATION

Allocate only the fees realised in the call and keep pre-existing balances out of it:

```
 (uint256 fees0, uint256 fees1) = _collectFees();
if (fees0 == 0 && fees1 == 0) revert InsufficientAmount();
uint256 residual0 = TOKEN_0.balanceOf(address(this)) - fees0;
uint256 residual1 = TOKEN_1.balanceOf(address(this)) - fees1;
// swap within the fee batch only, then:
(uint256 amount0, uint256 amount1) = _distributeTokenFees(feeBatch0, feeBatch1);
(uint128 liquidityAdded, , ) = _increaseLiquidity(amount0 + residual0, amount1 + residual1,
lpParams);
```

Compute the protocol compound slice from the fee-derived portion only and credit the unconsumed `used0` / `used1` in `protocolDepositE280` to `protocolOwnedLiquidity` instead of leaving them in `balanceOf`.

MEDIUM SEVERITY · 2 FINDINGS

M-01 MEDIUM ACKNOWLEDGED

A fee recipient that cannot receive a transfer permanently bricks `sunset`, locking all protocol-owned liquidity

`E280ExternalLpVault.sol#_distributeProtocolExit()` is only reached from `sunset()` and it pushes to two fixed recipients with no failure handling and no per-leg guard:

```
 token.safeTransfer(genesisAddress, genesisAmount);
token.safeTransfer(BUY_BURN_ADDRESS, buyBurnAmount);
```

If either can't receive the token, `sunset()` reverts. That matters because `sunset()` is the only path that converts `protocolOwnedLiquidity` back into tokens.

The two recipients differ in how recoverable they are:

- `BUY_BURN_ADDRESS` blocked - `compound` / `rebalance`: recoverable, `setAllocationBps(0, x)` zeroes `buyBurnBps`, skipping the `_distributeTokenFees` leg. `sunset`: bricked, `_distributeProtocolExit` hardcodes the buy-burn share at `amount - genesisAmount`.
- `genesisAddress` blocked - `compound` / `rebalance`: bricked, `GENESIS_BPS` is a `constant`, so the leg can't be zeroed. `sunset`: bricked.

There is no escape either way. `setGenesisAddress()` is gated on `msg.sender == genesisAddress`, so the owner can't repoint the genesis role. `BUY_BURN_ADDRESS` is `immutable` with no setter. `rescueERC20()` rejects both pool tokens and E280.

Two triggers, neither needing the owner. A blocklist-capable pool token blacklisting either address and USDC and USDT are the most plausible legs for an LP vault. Or the genesis key holder pointing the role at an address that can't receive the token, since `setGenesisAddress` only validates that it's non-zero.

IMPACT

The protocol entire accumulated position is locked and all accrued fees are lost with it. Every `_collectFees()` call site sits downstream of a distribution and `_removeLiquidity` deliberately caps its `collect` at the removed principal, so fees left on the NFT can't be reached by any other path.

Rated Medium rather than High because it needs an external precondition: a blocklisting event or a genesis holder acting against their own interest. Share holders are never trapped. `withdraw()` in both branches touches only the position manager and the two pool tokens, so principal always exits.

RECOMMENDATION

Make the payouts failure-isolated so a third party state can't halt the vault:

```
function _payout(IERC20 token, address to, uint256 amount) internal {
   if (amount == 0) return;
   try token.transfer(to, amount) returns (bool ok) {
       if (!ok) pendingPayout[address(token)][to] += amount;
   } catch {
       pendingPayout[address(token)][to] += amount;
   }
}
```

M-01 · CONTINUED

Add a `claimPayout(token)` for recipients and per-leg zero guards in `_distributeProtocolExit` to match the ones already in `_distributeTokenFees`.

M-02 MEDIUM ● ACKNOWLEDGED

Share price excludes value the vault already controls, so a deposit placed one transaction before any realisation event captures a share of it

A share is priced purely on position liquidity. `E280ExternalLpVault.sol#getTotalLiquidity()` returns `positions(nftId).liquidity` and `_deposit()` prices against it:

```
 uint256 userLiquidityBefore = getTotalLiquidity() - protocolOwnedLiquidity;
...
userShares = Math.mulDiv(userLiquidity, totalSharesBefore, userLiquidityBefore);
```

Two pools of value the vault controls are missing from that figure: **uncollected Uniswap fees** and **idle token balances**. Each one gets injected into share value in a single step, by a public transaction, minting no shares. Whoever holds shares at that instant takes a pro-rata cut of value earned or funded before they arrived. There's no deposit lock, no cooldown and the pending amounts are readable on-chain, so an attacker only acts when it pays.

Three triggers. `compound()` realises the fee batch. `rebalance()` does the same in its first two steps. `sunset()` is the biggest, because it does three things at once: realises the final batch, switches the valuation basis from position liquidity to `TOKEN_x.balanceOf(address(this))` which includes every idle token and which `_distributeProtocolExit` never touches and flips `withdraw()` to a branch that charges no exit fee.

IMPACT

`compound()`, break-even against the pending batch with user liquidity at 57,587:

- Pending batch 492 (0.85% of user liquidity): attacker net -170
- Pending batch 1,954 (3.4% of user liquidity): attacker net +209
- Pending batch 14,655 (25.4% of user liquidity): attacker net +3,489

So it pays once the batch clears roughly 1-3% of user liquidity. An honest holder measurably loses: Alice payout drops from 10,404 to 10,150 from one attacked compound, a 2.4% loss. `compound()` has no cooldown and no enforced cadence, so how often the batch sits above break-even is purely operational.

`rebalance()`: +489 for the attacker, Alice down 1.8%.

`sunset()`, with the attacker depositing 60,000 of each token:

- Idle balance 0, no fees: attacker net -600, the entry fee only, since the exit fee is genuinely absent
- Idle balance 0, fee batch only: attacker net +1,189
- Idle balance 18,115 from one sane out-of-range protocol deposit: attacker net +8,059
- Idle balance 30,000 donated: attacker net +19,400, which is 64.7% of it and exactly the attacker share of supply

In range with a well-chosen keeper ratio the residue is near zero and the sunset payoff drops to +0.24-0.38%. The out-of-range case is what makes it material and that ordinary V3 behaviour rather than an error.

The sunset variant is also the hardest to defend. The attacker only needs to hold shares at the instant of sunset. After that the value is frozen in token balances and they can withdraw whenever they like with no price risk. `setPaused(true)` does block the deposit, but front-running the pause transaction gives the identical result because withdrawals are never pausable. Only a pause landing far enough ahead to impose real impermanent-loss risk degrades it and nothing in the code or NatSpec prescribes that.

RECOMMENDATION

M-02 · CONTINUED

Make the share price reflect everything the vault controls. Read `feeGrowthInside0LastX128` / `feeGrowthInside1LastX128` and `tokensOwed0` / `tokensOwed1` from `positions(nftId)`, convert to liquidity-equivalent units and add them plus the idle token balances to `userLiquidityBefore` in `_deposit()` and to the redemption base in `withdraw()`. Entering and exiting then price against the same quantity a realisation event releases.

LOW SEVERITY · 4 FINDINGS

L-01 LOW ACKNOWLEDGED

`rebalance` removes 100% of its own liquidity before swapping through the same pool

`E280ExternalLpVault.sol#rebalance()` removes the entire position at step 4, then swaps at step 5. `_swapTokenForToken()` routes through the vault own pool, since the path is built with `V3_POOL.fee()`. By the time the swap runs the vault is no longer an LP in the book it's trading into, so it pays the full pool fee to competing LPs instead of recapturing its share, and its own missing liquidity amplifies the price impact.

`compound()` uses the opposite ordering and swaps while the position is still deployed, which is what makes this an oversight rather than a design choice.

IMPACT

Scales with the vault share of in-range liquidity: 814 bps of the swap amount when the vault is roughly half the pool, 83 bps at ~20%, 5 bps at ~5%. Only fires when `p.swapAmount > 0`. Material if the vault is meant to be a dominant LP in its pair, negligible otherwise.

RECOMMENDATION

Move the swap before `_removeLiquidity`.

L-02 LOW ● FIXED

`depositE280` credits the amount requested rather than the amount received and the tax-exemption requirement is undocumented

```
 E280.safeTransferFrom(msg.sender, address(this), e280Amount);
(uint256 amount0, uint256 amount1) = _splitAndSwapE280(e280Amount, ...);
```

The vault swaps `e280Amount`, not what actually arrived and `_swapE280ForToken` uses `swapExactTokensForTokens` rather than the fee-supporting variant. Both only work if the vault is exempt from any E280 transfer tax, which is an undocumented hard deployment prerequisite. The repository's own `T280.sol` models E280 with a 4% tax and a whitelist, which is why this is flagged.

IMPACT

If exemption is missed or later revoked while the V2 pair stays exempt, the shortfall is drawn from the E280 accumulated for `protocolDepositE280` and the path is cheaper for the depositor than doing the swap themselves. That makes it a drain users are incentivised to run, not just a loss they suffer. If neither is exempt, the V2 K check fails and the whole E280 flow reverts.

RECOMMENDATION

Measure the balance delta and swap that:

```
 uint256 before = E280.balanceOf(address(this));
E280.safeTransferFrom(msg.sender, address(this), e280Amount);
uint256 received = E280.balanceOf(address(this)) - before;
(uint256 amount0, uint256 amount1) = _splitAndSwapE280(received, ...);
```

L-03 LOW ● ACKNOWLEDGED

E280 received after `sunset` is permanently locked and the address is a constant across a multi-chain deploy config

`sunset()` pays out the E280 balance once, then closes every remaining path. `protocolDepositE280` is sunset-gated, `depositE280` reverts, the sunset `withdraw` branch handles only the pool tokens and `rescueERC20()` explicitly rejects E280. The vault is a standing recipient of E280, so anything in flight or arriving later is stuck.

Separately, `E280` is a hardcoded `constant` while every other external address is a constructor parameter and `deploy/constants.ts` carries filled-in router addresses for ethereum, bsc and arbitrum. If E280 isn't deployed at that same address on a target chain, the high-level call in `E280.balanceOf(address(this))` hits an address with no code and reverts, so `sunset()` bricks and the vault has no exit at all.

IMPACT

Permanent lock of post-sunset E280 inflow. On a chain where E280 sits at a different address, permanent inability to shut the vault down.

RECOMMENDATION

Let `rescueERC20` move E280 once `vaultSunset` is true and make the address an immutable constructor parameter rather than a constant.

L-04 LOW ● ACKNOWLEDGED

The 300 bps `depositE280` fee is bypassable at 50 bps

`depositE280` charges `depositE280ProtocolFeeBps` (300) where `deposit` charges `depositProtocolFeeBps` (50). The end state is reachable either way. A user can perform the two E280 swaps themselves on the same V2 router, then call `deposit(amount0, amount1, ...)` and pay 50 bps for an identical position.

IMPACT

On a $1,000-equivalent deposit the bypass saves $25 for the cost of one extra transaction, so it clears gas at any meaningful size. Anyone who notices routes around the premium tier, which means expected revenue from the E280 on-ramp is close to zero while it still carries the integration risk of the V2 path.

RECOMMENDATION

Decide which the fee is for. If it prices the convenience of the bundled swaps, it has to be close enough to 50 bps that unbundling isn't worth a transaction. If it prices something else, that needs to be enforced rather than left to the caller's choice of entry point.

AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

@blckhv

REPORT

E280 ExternalLp · August 2026